



Universidade de São Paulo - São Carlos
Instituto de Ciências Matemáticas e de Computação

SCC-241– Laboratório de Bases de Dados

Projeto Final – P4
Desenvolvimento da aplicação completa

Docente: Caetano Traina Jr. — caetano@icmc.usp.br

PAE: João Victor Cosme Neres de Sousa — joaovneres@usp.br 1º Semestre de 2026

Data para entrega: 22 de junho de 2026

Datasets

Base de dados da Fórmula 1 - FIA, com cidades e aeroportos do mundo.

A base de dados a ser utilizada é a mesma estruturada ao longo das atividades da disciplina. Os arquivos necessários para a carga de dados estão disponíveis na **Seção 3 do Moodle (e-Disciplinas)**, podendo ser acessados no link a seguir: [link de acesso](#).

Os dados disponibilizados foram ligeiramente modificados para facilitar o desenvolvimento da atividade. No entanto, caso desejem, vocês podem utilizar os dados originais, por exemplo, para incluir informações atualizadas das últimas corridas.

Nesse caso, os dados podem ser obtidos nos seguintes *sites*:

- Dados da Fórmula-1: [Formula 1 Championships \(1950-2025\) – Kaggle](#)
 - Países e Cidades do planeta: [GeoNames](#)
 - Aeroportos: [OurAirports](#)
-

Atividades do Projeto Final

O objetivo deste trabalho é integrar os conhecimentos estudados ao longo da disciplina por meio do desenvolvimento de um protótipo de aplicação completa, capaz de manipular dados, executar consultas, gerar relatórios e apresentar informações de forma organizada e intuitiva ao usuário.

A aplicação deverá ser centrada na exploração da **Base de Dados da Fórmula 1 - FIA**, utilizando uma interface amigável e recursos de banco de dados trabalhados durante o semestre.

Instruções e Orientações Gerais

No desenvolvimento da ferramenta, os seguintes pontos devem ser observados:

- A implementação deve considerar a versão da base estruturada, carregada e corrigida ao longo das atividades anteriores, especialmente após as etapas de normalização, deduplicação e ajuste de vínculos realizadas no Trabalho Prático T1.
 - As informações devem ser apresentadas ao “usuário da ferramenta” de forma intuitiva. Os nomes das colunas exibidas em telas, tabelas, dashboards e relatórios devem estar inteligíveis em Língua Portuguesa.
 - A tecnologia utilizada para o desenvolvimento da interface é de livre escolha do grupo, desde que permita implementar as funcionalidades solicitadas e demonstrar claramente a interação com o banco de dados.
 - A ferramenta deve atender a três tipos de usuários:
 - **Administrador do sistema:** existe apenas um, identificado como `admin`;
 - **Escuderia:** deve existir um usuário para cada escuderia armazenada na tabela `CONSTRUCTORS`, identificado pelo padrão `<constructor_ref>_c`;
 - **Piloto:** deve existir um usuário para cada piloto armazenado na tabela `DRIVERS`, identificado pelo padrão `<driver_ref>_d`.
 - Algumas das informações solicitadas já foram trabalhadas em atividades anteriores, mas deverão ser integradas agora em uma aplicação única.
 - Os comandos SQL utilizados pela aplicação devem estar explícitos no código. Não devem ser utilizadas ferramentas que automatizem ou ocultem os *scripts* executados, impedindo sua análise durante a avaliação.
 - Devem estar destacados, nos respectivos códigos, os conceitos estudados ao longo da disciplina, com comentários justificando seu uso, incluindo:
 - procedimentos e funções;
 - *triggers*;
 - visões;
 - índices;
 - consultas com junções, agregações e filtros;
 - controle de acesso e autenticação, quando aplicável.
-

1 - Administrar Usuários

O acesso à ferramenta deve ser feito somente a partir de uma tela inicial de *login*, na qual cada usuário deve ser autenticado para acessar as funcionalidades disponíveis ao seu tipo de usuário.

Para simplificar o desenvolvimento e os testes do protótipo, os nomes de usuário e senha devem seguir os padrões definidos a seguir:

Admin: Pode acessar qualquer informação da base.

Login: `admin`

Senha: `admin`

Escuderia: Pode acessar somente as informações relativas à sua escuderia e aos pilotos que correm ou correram por ela.

Login: `<constructor_ref>_c`

Senha: <constructor_ref>

Exemplo: para a escuderia `mclaren`, o login deve ser `mclaren_c` e a senha deve ser `mclaren`.

Piloto: Pode acessar somente informações relativas ao seu próprio desempenho.

Login: <driver_ref>_d

Senha: <driver_ref>

Exemplo: para o piloto `hamilton`, o login deve ser `hamilton_d` e a senha deve ser `hamilton`.

Pontos que precisam ser tratados:

1. Deve ser criada uma tabela chamada `USERS`, contendo, no mínimo, os seguintes atributos:

`userid, login, password, tipo, id_original`

O atributo `login` deve ser único. O atributo `id_original` deve armazenar o identificador do registro correspondente na tabela de origem, isto é, o identificador do piloto ou da escuderia. Para o usuário administrador, esse atributo pode ficar nulo.

2. A senha dos usuários deve ser armazenada de forma protegida. Caso a implementação utilize usuários reais do SGBD PostgreSQL, a autenticação deve ser configurada com `SCRAM-SHA-256`. Caso a autenticação seja feita diretamente pela tabela `USERS`, não deve ser armazenada senha em texto puro.
3. Cada usuário deve pertencer a apenas um dos seguintes tipos:

`'Admin', 'Escuderia' ou 'Piloto'`

4. Os pilotos e escuderias já cadastrados na base da Fórmula 1 deverão ser cadastrados também na tabela `USERS`, seguindo os padrões de *login* e senha definidos anteriormente.
5. Deve-se assegurar que, sempre que um piloto ou escuderia for criado ou modificado na respectiva tabela, o registro correspondente na tabela `USERS` seja criado ou atualizado automaticamente.
6. Deve ser criada uma tabela chamada `USERS_LOG`, destinada a auditar as atividades de acesso ao sistema, incluindo *login* e *logout*. Cada registro deve conter, no mínimo:
 - o `userid` do usuário;
 - o tipo da ação realizada, por exemplo `'LOGIN'` ou `'LOGOUT'`;
 - a data e hora da ação.

2 - Fluxo de Telas da Ferramenta

A estrutura da ferramenta deve estar centrada em três telas principais, descritas a seguir. Cada tela deve apresentar variações de acordo com o tipo de usuário autenticado.

Tela 1: Tela de Login. Solicita a identificação do usuário e sua senha. Após a confirmação do *login*, deve ser apresentada a Tela 2.

Tela 2: Tela de Dashboard. Apresenta informações sumarizadas de acordo com o tipo de usuário logado e deve funcionar como a tela principal de navegação da ferramenta. Em todas as variações, deve apresentar:

- o nome ou identificação do usuário logado;
- as informações de *dashboard* correspondentes ao tipo de usuário;
- botões ou links para as ações disponíveis ao tipo de usuário autenticado;

- caminho para a Tela 3, destinada aos relatórios.

De acordo com o tipo de usuário, a tela deve apresentar:

- **Admin:** nome do usuário e destaque para sua identificação como administrador;
- **Escuderia:** nome da escuderia e quantidade de pilotos associados a ela;
- **Piloto:** nome da escuderia associada e nome completo do piloto.

Tela 3: Tela de Relatórios. Deve apresentar botões ou recursos equivalentes para solicitar os relatórios disponíveis ao tipo de usuário logado. Sempre que um relatório for solicitado, a tela deve apresentar o resultado correspondente. Após o encerramento da visualização de um relatório, a ferramenta deve retornar à Tela 3.

3 - Ações Disponibilizadas aos Usuários

As ações disponibilizadas dependem do tipo de usuário autenticado.

Admin:

- **Cadastrar escuderias:** exibe uma janela ou formulário que permite inserir os dados necessários para adicionar uma nova tupla na tabela `CONSTRUCTORS`. Os dados a serem informados são:

`constructor_ref`, `name`, `country_id` e `wikipedia_url`

- **Cadastrar pilotos:** exibe uma janela ou formulário que permite inserir os dados necessários para adicionar um novo piloto na tabela `DRIVERS`. Os dados a serem informados são:

`driver_ref`, `given_name`, `family_name`, `date_of_birth` e `country_id`

- Quando houver um novo cadastro de escuderia ou piloto, o sistema deverá inserir automaticamente o respectivo usuário na tabela `USERS`, utilizando *triggers* e seguindo os padrões de *login* e senha definidos anteriormente.
- Caso já exista algum usuário com o *login* gerado, a *trigger* deve cancelar a operação e impedir a inserção inconsistente na tabela de origem.

Escuderia:

- **Consultar piloto por sobrenome:** exibe uma janela ou formulário que permite indicar o sobrenome de um piloto. O programa deve verificar se há algum piloto com esse sobrenome que já tenha corrido pela escuderia logada. Caso exista, a ferramenta deve apresentar o nome completo, a data de nascimento e o país ou a nacionalidade associada ao piloto.

Dica: para verificar se um piloto já correu por uma escuderia, consulte a tabela `RESULTS`.

- **Inserir novos pilotos por arquivo:** exibe uma janela ou formulário que permite indicar o nome de um arquivo, acessível no sistema operacional, contendo informações de um ou mais pilotos.
 - Cada linha do arquivo deve conter as informações de um piloto.
 - Cada piloto deve ter indicado no arquivo:

`driver_ref`, `given_name`, `family_name`, `date_of_birth` e `country_id`

Antes da inserção, deve ser verificado que não exista outro piloto com o mesmo nome e sobrenome. Caso o piloto já exista, isso deve ser informado ao usuário e a inserção deve

ser cancelada.

A inserção do piloto deve criar o registro correspondente na tabela [DRIVERS](#) e o respectivo usuário na tabela [USERS](#). Caso o grupo opte por registrar explicitamente a associação entre o novo piloto e a escuderia logada, essa decisão deve ser descrita no relatório e implementada de forma compatível com o esquema relacional adotado.

Piloto:

- Usuários do tipo Piloto não podem alterar dados da base. Eles podem apenas visualizar os relatórios e o *dashboard* referentes ao próprio piloto.
-

4 - Definição da Tela de *Dashboard*

Cada tipo de usuário deve possuir seu próprio *dashboard*, com informações específicas ao seu perfil.

Admin:

1. Quantidade total de pilotos, escuderias e temporadas cadastradas.
2. Lista das corridas cadastradas na temporada mais recente da base, com circuito, data, horário e quantidade de voltas registrada nos resultados.
3. Lista das escuderias que correram na temporada mais recente da base, cada uma com o total de pontos obtidos.
4. Lista dos pilotos que correram na temporada mais recente da base, cada um com o total de pontos obtidos.

Escuderia: Devem ser criadas funções ou procedimentos armazenados que recebam dados da escuderia como parâmetro e retornem as seguintes informações:

1. quantidade de vitórias da escuderia, considerando as corridas em que obteve a primeira posição;
2. quantidade de pilotos diferentes que já correram pela escuderia;
3. primeiro e último ano em que há dados da escuderia na base, considerando a tabela [RESULTS](#).

Piloto: Devem ser criadas funções ou procedimentos armazenados que recebam dados do piloto como parâmetro e retornem as seguintes informações:

1. primeiro e último ano em que há dados do piloto na base, considerando a tabela [RESULTS](#);
 2. para cada ano em que o piloto competiu e para cada circuito em que correu:
 - quantidade de pontos obtidos;
 - quantidade de vitórias, considerando as corridas em que obteve a primeira posição;
 - quantidade total de corridas em que participou.
-

5 - Relatórios

Os relatórios devem ser apresentados de forma compreensível ao respectivo tipo de usuário. Recomenda-se aplicar ordenações, filtros e nomes de colunas que facilitem a interpretação dos resultados.

Os índices criados para auxiliar os relatórios devem ser indicados no código e justificados brevemente no relatório final, explicando quais filtros, junções ou ordenações eles procuram otimizar.

Admin:

- **Relatório 1:** Indica a quantidade de resultados por *status*, apresentando o nome do *status* e sua respectiva contagem.
- **Relatório 2:** Recebe o nome de uma cidade e, para cada cidade brasileira que tenha esse nome, apresenta todos os aeroportos brasileiros que estejam a, no máximo, 100 km da respectiva cidade e que sejam dos tipos '[medium_airport](#)' ou '[large_airport](#)'. O relatório deve apresentar:
 - nome da cidade pesquisada;
 - código IATA do aeroporto;
 - nome do aeroporto;
 - cidade em que o aeroporto está localizado;
 - distância entre a cidade pesquisada e o aeroporto;
 - tipo do aeroporto.

Deve ser criado também um índice que auxilie essa consulta.

- **Relatório 3:** Lista todas as escuderias cadastradas, cada uma com a respectiva quantidade de pilotos, e gera um relatório hierárquico em três níveis:
 1. quantidade de corridas cadastradas no total;
 2. quantidade de corridas cadastradas por circuito, com quantidade mínima, média e máxima de voltas registradas nos resultados;
 3. para cada corrida por circuito, indica a respectiva quantidade de voltas registradas e a quantidade de pilotos participantes.

Escuderia: Recomenda-se a criação de funções ou procedimentos armazenados para cada relatório, recebendo como parâmetro o identificador da escuderia logada.

- **Relatório 4:** Lista os pilotos da escuderia e a quantidade de vezes em que cada um alcançou a primeira posição em uma corrida. Os pilotos devem ser identificados por seu nome completo. Devem ser criados os índices necessários para auxiliar essa consulta.

Dica: para verificar se um piloto já correu por uma escuderia e se houve vitória, consulte a tabela [RESULTS](#).

- **Relatório 5:** Lista a quantidade de resultados por *status*, apresentando o *status* e sua contagem, limitada ao escopo da escuderia logada.

Piloto: Recomenda-se a criação de funções ou procedimentos armazenados para cada relatório, recebendo como parâmetro o identificador do piloto logado.

- **Relatório 6:** Consulta a quantidade total de pontos obtidos por ano de participação na Fórmula 1, apresentando, para cada ano, as corridas em que os pontos foram obtidos. As informações devem estar restritas apenas ao piloto logado. Devem ser criados os índices necessários para auxiliar essa consulta.
- **Relatório 7:** Lista a quantidade de resultados por *status* nas corridas em que o piloto participou, apresentando o *status* e a contagem de cada um, limitada ao escopo do piloto logado.

Entrega Final

Cada equipe deverá entregar dois arquivos no **Moodle**:

- Um arquivo no formato **.zip**, contendo:
 - o código-fonte da aplicação;
 - os *scripts* SQL desenvolvidos;
 - os arquivos necessários para executar o protótipo;
 - um arquivo **README** com instruções de execução da aplicação e dos *scripts*.
- Um único relatório sucinto, no formato **.pdf**, contendo:
 - a descrição das funcionalidades implementadas;
 - as técnicas de banco de dados utilizadas;
 - os índices, funções, visões e *triggers* criados;
 - as principais decisões tomadas pelo grupo;
 - exemplos de uso da aplicação, preferencialmente com capturas de tela;
 - as dificuldades encontradas e como foram tratadas.

O código deve conter comentários que auxiliem o entendimento da implementação, principalmente nos trechos relacionados aos conceitos trabalhados na disciplina.

Na apresentação do projeto, poderão ser feitas perguntas individuais aos membros do grupo. Essas perguntas irão compor a nota individual e poderão abordar qualquer parte do trabalho.

Todos os membros devem ser capazes de explicar sua contribuição no desenvolvimento do projeto como um todo, não apenas em uma parte isolada.

Os principais critérios avaliados serão:

- funcionalidades implementadas;
- uso adequado de SQL e dos conceitos estudados na disciplina;
- índices criados para funcionalidades relevantes;
- funções, visões e *triggers* implementados;
- organização e clareza do código;
- usabilidade do sistema;
- correteza das soluções;
- clareza das justificativas apresentadas no relatório.

Atenção

- O relatório deve evidenciar claramente as decisões adotadas pelo grupo e justificar as escolhas realizadas no desenvolvimento do trabalho.
- Não serão aceitos projetos feitos à mão e a organização clara das respostas também é um

ponto avaliado.

- Plágio será avaliado com nota zero.
- Os arquivos devem ser submetidos:

até às **08:00** do dia **22 de junho de 2026**
com a **postagem apenas no moodle**
e-Disciplinas.

Bom Trabalho!